
Ficha de exercícios 4 – Codificação de dados em informação visual

1. Pretende-se analisar, explorar e visualizar o Iris dataset, que reúne as características de flores da espécie “iris”, nomeadamente tamanho e comprimento de pétala e sépala. Mais informações pode ser encontradas aqui:

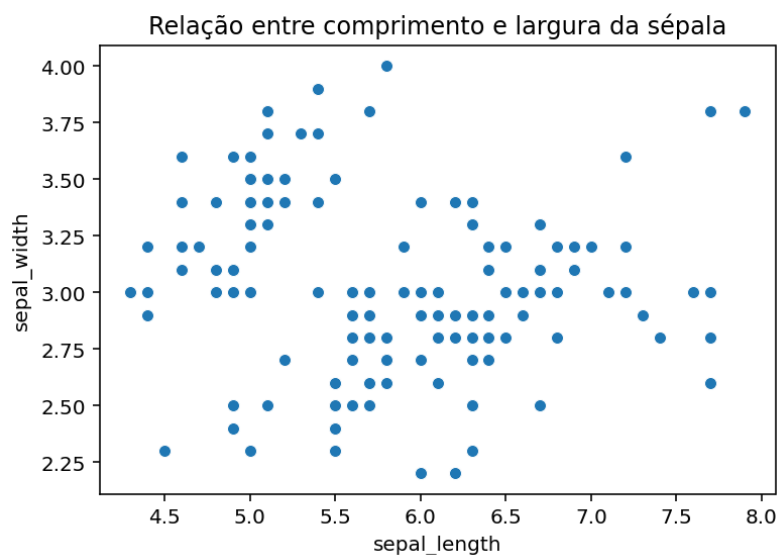
<https://archive.ics.uci.edu/dataset/53/iris>.

- a. Experimente criar um gráfico de dispersão com os elementos `x="sepal_length", y="sepal_width"`.

Código (precisa do seaborn [sns] e do matplotlib.pyplot [plt]):

```
iris = sns.load_dataset("iris")  
  
plt.figure(figsize=(6,4))  
sns.scatterplot(data=iris, x="sepal_length", y="sepal_width")  
plt.title("Relação entre comprimento e largura da sépala")  
plt.show()
```

Resultado esperado:

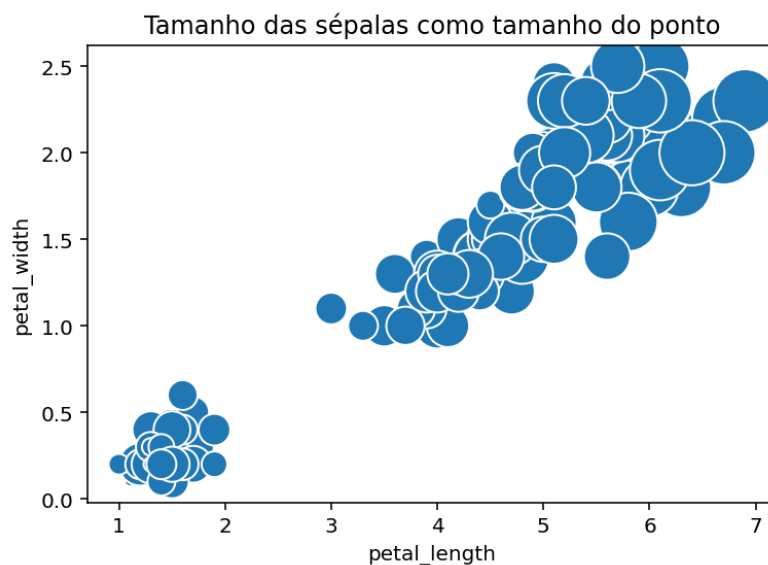


O que se pode concluir acerca da dispersão apresentada?

- b. Agora, corra o seguinte código para verificar a associação entre a marca “ponto” e dimensão orientada pela característica “sepal_length”.

```
sns.scatterplot(  
    data=iris,  
    x="petal_length",  
    y="petal_width",  
    size="sepal_length",  
    sizes=(20, 200),  
    legend=False  
)  
plt.title("Tamanho das sépalas como tamanho do ponto")  
plt.show()
```

Resultado esperado:



Qual lhe parece ser a relação entre as características petal_width, petal_length e sepal_length? Podemos estabelecer alguma ordem de relação visual, ainda que subjetiva?

- c. Vamos agora explorar o método de remoção de *outliers* (valores “salientes” que saem do padrão de dados), aplicando a técnica do intervalo interquartil (IQR), que materializa através do seguinte conjunto de passos:

- i. Ordenar dados: Organize todos os dados do menor para o maior.
- ii. Encontrar o primeiro quartil (Q1).
- iii. Encontrar o terceiro quartil (Q3).
- iv. Determinar o IQR, através da fórmula $IQR = Q3 - Q1$.
- v. Identificar os pontos de dados que estão abaixo de $Q1 - (1.5 \times IQR)$ ou acima de $Q3 + (1.5 \times IQR)$. Esses serão considerados os *outliers*.

Código (precisa do Pandas):

```
Q1 = iris["sepal_width"].quantile(0.25)
Q3 = iris["sepal_width"].quantile(0.75)
IQR = Q3 - Q1

# filtro de outliers
filtro = (iris["sepal_width"] >= Q1 - 1.5*IQR) & (iris["sepal_width"] <= Q3 + 1.5*IQR)
iris_limpo = iris[filtro]

print(f"Removidos {len(iris) - len(iris_limpo)} outliers.")
```

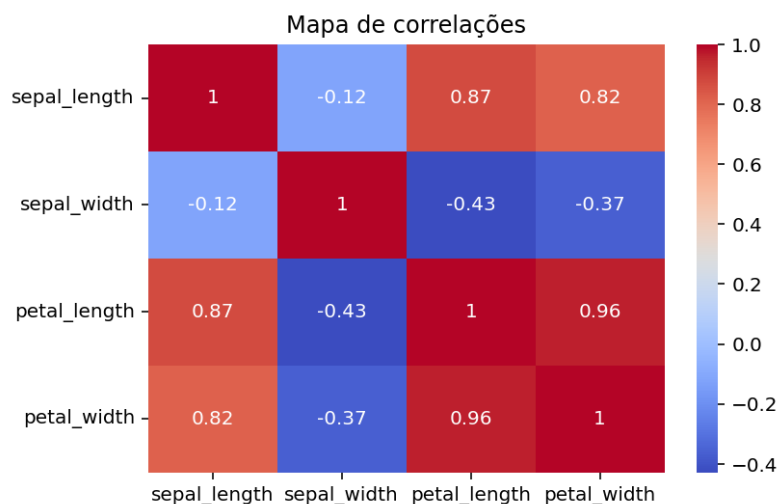
O resultado é uma mensagem na consola:

“Removidos 4 outliers.”

- d. Vamos agora investigar correlações entre características, através do seguinte bloco de código:

```
corr = iris.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Mapa de correlações")
plt.show()
```

Resultado esperado:



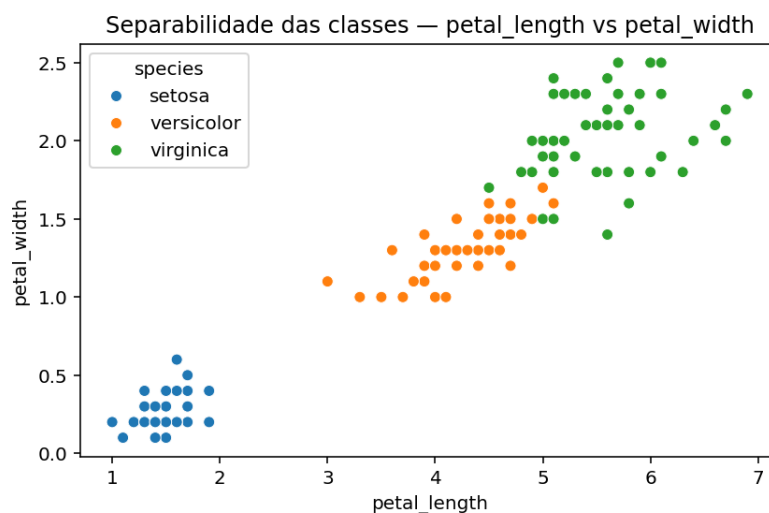
Sabendo que quanto mais “vermelho” e próximo de 1 for a célula, maior a correlação cruzada (como verificável no conjunto diagonal de células), identifique quais as características que melhor estabelecem correlação.

- e. Agora, vamos tentar perceber se as classes de iris são bem separáveis com base na comparação das suas características, nomeadamente `petal_length` e `petal_width`. Para tal, corra o seguinte código:

```
plt.figure(figsize=(6,4))
sns.scatterplot(
    data=iris,
    x="petal_length",
    y="petal_width",
    hue="species",
    style="species"
)

plt.title("Separabilidade das classes — petal_length vs petal_width")
plt.xlabel("petal_length")
plt.ylabel("petal_width")
plt.tight_layout()
plt.show()
```

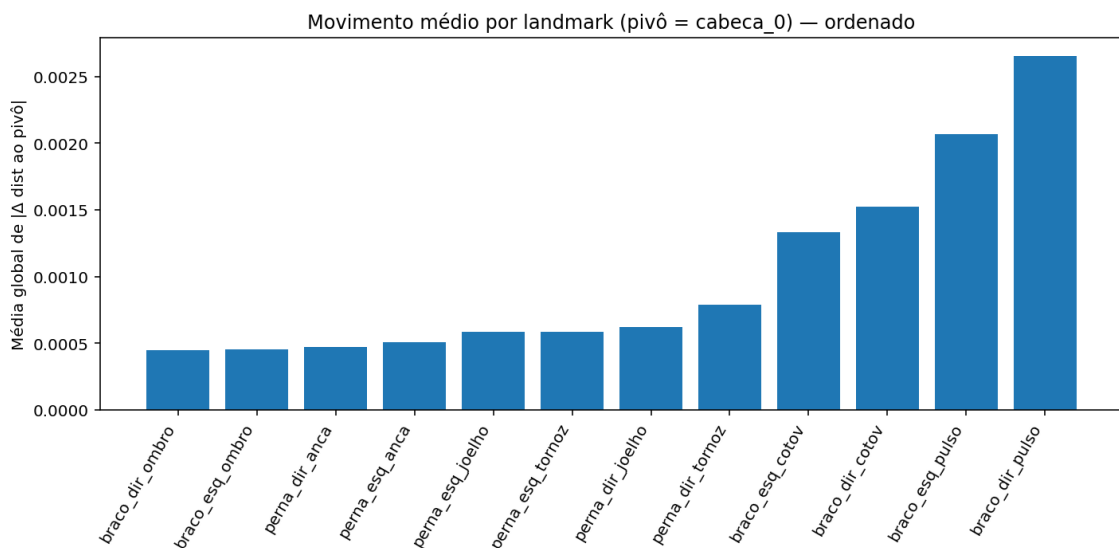
Resultado esperado:



Experimente outras combinações de características: `sepal_length` e `sepal_width`, `petal_length` e `sepal_length`, `petal_width` e `sepal_width`, `petal_length` e `sepal_width`, `petal_width` e `sepal_length`. Compare os resultados com o mapa de correlações da alínea d). É possível retirar algumas ilações?

2. Vamos explorar os gráficos de barras, mas recorrendo a *prompts* colocados à IA generativa (e.g. ChatGPT). Considere o ficheiro “landmarks_pose.csv” disponibilizado na BlackBoard com esta ficha, que foi gerado com base na análise de 6 vídeos de guarda-redes de andebol em treino.

- a. No sumário das partes do corpo que foram mais movimentadas para responder os remates, obteve-se o seguinte gráfico:

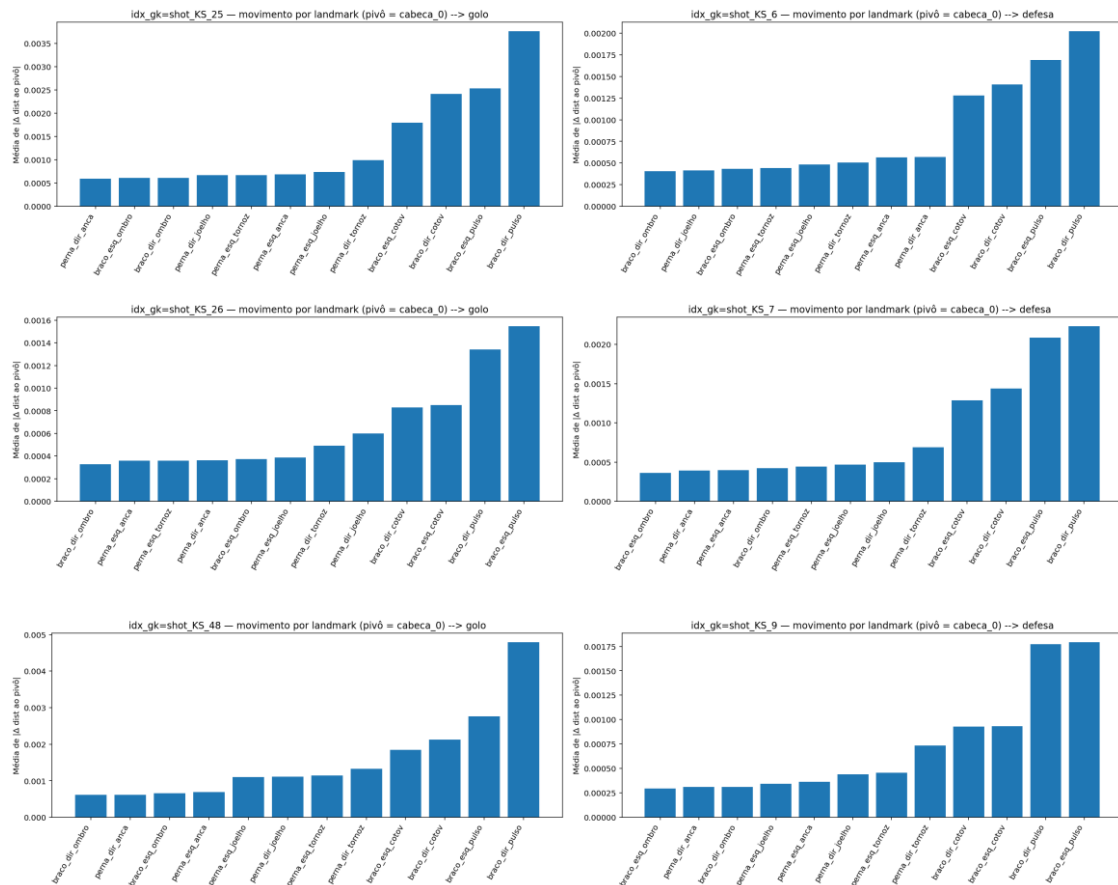


Utilize ferramentas gratuitas de modelos linguísticos generativos para solicitar a análise do ficheiro .csv e apoio no desenvolvimento do código que permite chegar ao gráfico atrás, porém, sem fornecer esse mesmo gráfico. Identifique e sistematize os requisitos do que pretende. Sugestão de indicações:

- Pretende-se realizar o sumário do movimento dos membros, visualizável em gráfico de barras ordenadas;
- A IA deve considerar, para cada “idx_gk”, realizar o cálculo da distância dos membros a um ponto de pivot (e.g. cabeça);
- A cada par de itens da tabela, devem medir-se as distâncias consecutivas (calculadas conforme o ponto anterior) e calcular o valor absoluto das suas diferenças e guardar esse resultado num somatório alusivo à *landmark* a ser medida.
- No final dos itens de cada “idx_gk”, deve fazer-se uma média das deslocações.

- Deve ser feita, também, uma média global das deslocações das *landmarks* entre todos os guarda-redes.
- Desse processamento devem resultar os dados a serem mostrado sob a forma de barras ordenadas de forma ascendente.

b. Ajuste o *prompt* para que a IA gere o código para apresentar o seguinte conjunto de gráficos desta forma:



Aspectos-chave a ter em conta:

- Agrupamento dos dados deve ser feito por guarda-redes;
- O código do passo anterior para cálculo dos membros mais mexidos deve ser replicado, mas individualizado ao guarda-redes;
- O resultado da ação do guarda-redes (golo ou defesa) deve ser apresentado no título do gráfico.

- c. Discutir com o modelo de linguagem formas alternativas de explorar os dados para obter informações de outros aspetos que possam ser relevantes. Por exemplo, simetria lateral:

```
🧠 Simetria lateral – guarda-redes shot_KS_25
```

	par	media_diferenca
3	perna_esq_anca ↔ perna_dir_anca	0.002962
0	braco_esq_ombro ↔ braco_dir_ombro	0.004023
4	perna_esq_joelho ↔ perna_dir_joelho	0.004184
5	perna_esq_tornoz ↔ perna_dir_tornoz	0.012661
1	braco_esq_cotov ↔ braco_dir_cotov	0.012843
2	braco_esq_pulso ↔ braco_dir_pulso	0.013540

3. Considere o ficheiro “athletes.csv”¹ guardado na BlackBoard, junto desta ficha.
- a. Inspeccione os campos (atributos) do dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("athletes.csv")

print(df.head())

print(df.info())
```

- b. Verifique qual a distribuição dos participantes por país, considerando o top-10 dos países com mais atletas.

¹ Retirado de <https://www.kaggle.com/datasets/rio2016/olympic-games>

```

counts = (
    df["nationality"]
    .astype(str)
    .str.strip()
    .value_counts()
    .sort_values(ascending=False)
)

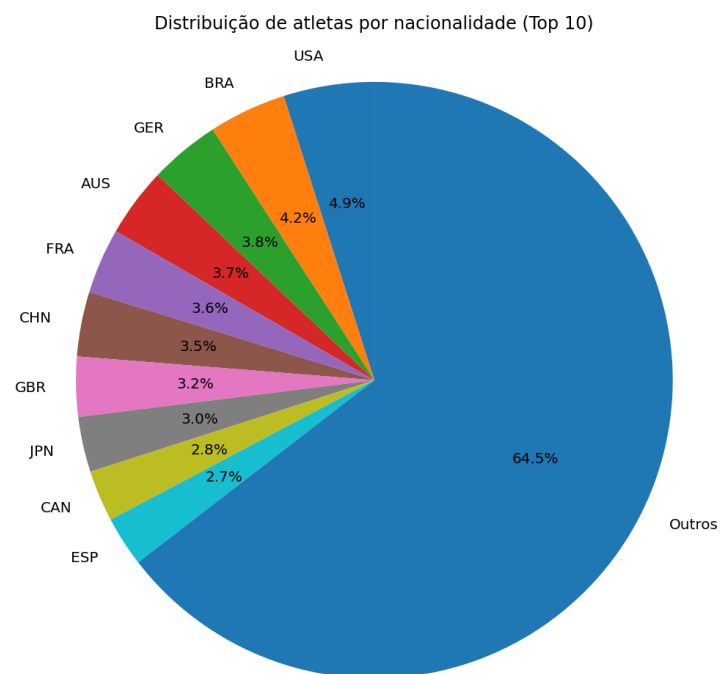
top_n = 10
counts_top = counts.head(top_n)
others_sum = counts.iloc[top_n:].sum()
if others_sum > 0:
    counts_top["Outros"] = others_sum

plt.figure(figsize=(7,7))
plt.pie(
    counts_top.values,
    labels=counts_top.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title(f"Distribuição de atletas por nacionalidade (Top {top_n})")
plt.axis("equal") # círculo perfeito
plt.tight_layout()
plt.show()

```

Resultado esperado:



- c. Usando gráfico de barras, verifique qual a distribuição de medalhas de ouro, prata e bronze arrecadas pelos EUA.


```

usa_df = df[df["nationality"].str.contains("usa", case=False, na=False)].copy()

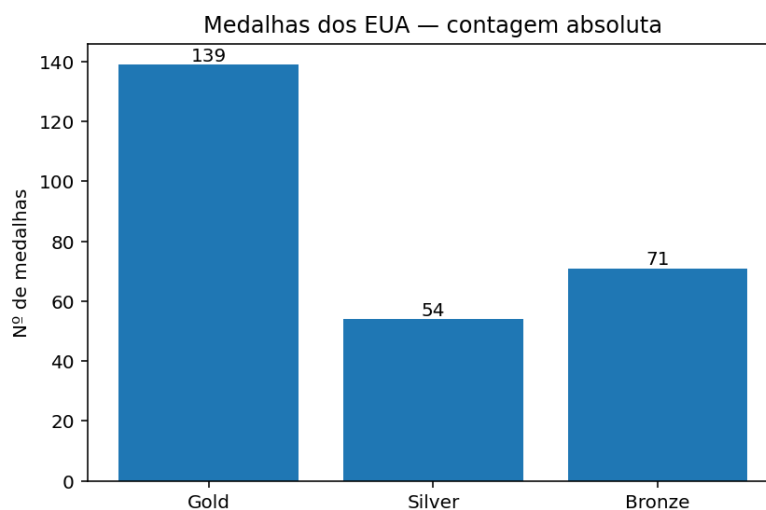
tot_gold = int(usa_df["gold"].fillna(0).sum())
tot_silver = int(usa_df["silver"].fillna(0).sum())
tot_bronze = int(usa_df["bronze"].fillna(0).sum())

labels = ["Gold", "Silver", "Bronze"]
values = np.array([tot_gold, tot_silver, tot_bronze], dtype=int)

plt.figure(figsize=(6,4))
plt.bar(labels, values)
for i, v in enumerate(values):
    plt.text(i, v, str(v), ha="center", va="bottom", fontsize=10)
plt.ylabel("Nº de medalhas")
plt.title("Medalhas dos EUA – contagem absoluta")
plt.tight_layout()
plt.show()

```

Resultado esperado:



- d. Agora, desenvolva um gráfico de radar para analisar a distribuição dos desportos em que os EUA arrecadaram o ouro, considerando o seguinte excerto de código.

```

TOP_N = 8

mask_usa = df["nationality"].astype(str).str.contains("USA", case=False, na=False)
usa = df[mask_usa].copy()

gold_per_sport = (
    usa.groupby("sport", as_index=True)["gold"]
        .sum()
        .sort_values(ascending=False)
)

gold_top = gold_per_sport.head(TOP_N)

# Preparar dados para o radar
labels = gold_top.index.tolist()
values_real = gold_top.values.astype(float)
values_norm = values_real / values_real.max() # para caber em [0,1] no radar

# fechar o círculo
angles = np.linspace(0, 2*np.pi, len(labels), endpoint=False).tolist()
angles += angles[:1]
rad_vals = values_norm.tolist() + [values_norm[0]]
rad_labels = labels + [labels[0]]

# Plot (radar)
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, polar=True)

# grelha radial (0..1)
ax.set_ylim(0, 1)
ax.set_yticklabels([]) # opcional: esconder ticks radiais
ax.set_xticks(angles[:-1])
ax.set_xticklabels(labels, fontsize=11)

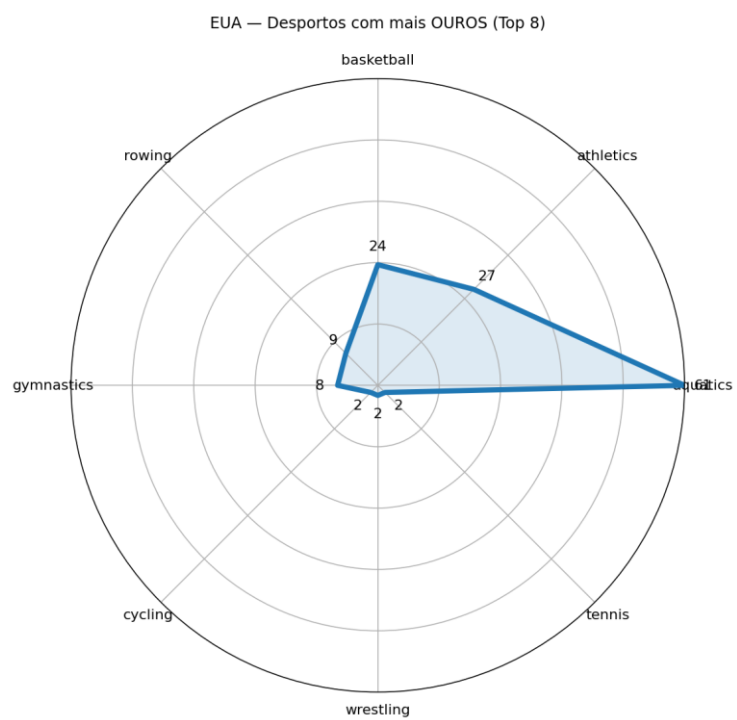
# linha/área
ax.plot(angles, rad_vals, linewidth=4)
ax.fill(angles, rad_vals, alpha=0.15)

# Anotações com as contagens reais junto aos pontos
for ang, val_n, val_real in zip(angles[:-1], values_norm, values_real):
    r = val_n + 0.06 # afastar um pouco para não sobrepor
    ax.text(ang, r, f"{int(val_real)}", ha="center", va="center", fontsize=11)

plt.title(f"EUA – Desportos com mais OUROS (Top {len(labels)})", pad=20)
plt.tight_layout()
plt.show()

```

Resultado esperado:



FIM